



Linear Regression

Predicting Continuous Values — The Math Behind the Best-Fit Line

What We're Covering Today



What is Regression, and how it differs from classification



The Cost Function — measuring how wrong the line is



Repeat & Convergence Theorem, and the Learning Rate



Model Evaluation — R^2 and Adjusted R^2



Ridge & Lasso Regression — regularization



Linear Regression — the problem statement & best-fit line



Gradient Descent — finding the minimum error



Multiple Linear Regression & the Hyperplane



Overfitting vs. Underfitting — bias & variance



Building it for real with scikit-learn



PART 1

What Is Regression?

Predicting a number, not a category



What Is Regression?

- Regression is a supervised learning task where the model predicts a continuous numeric value, instead of a category.
- Ask 'how much?' or 'how many?' and it's usually regression — price, temperature, salary, distance.
- This is the key difference from classification, which predicts a label (Yes/No, Spam/Not Spam) instead of a number.
- Linear Regression is the simplest and most fundamental regression algorithm — the natural starting point for ML.



**Regression →
numbers
Classification →
categories**



PART 2

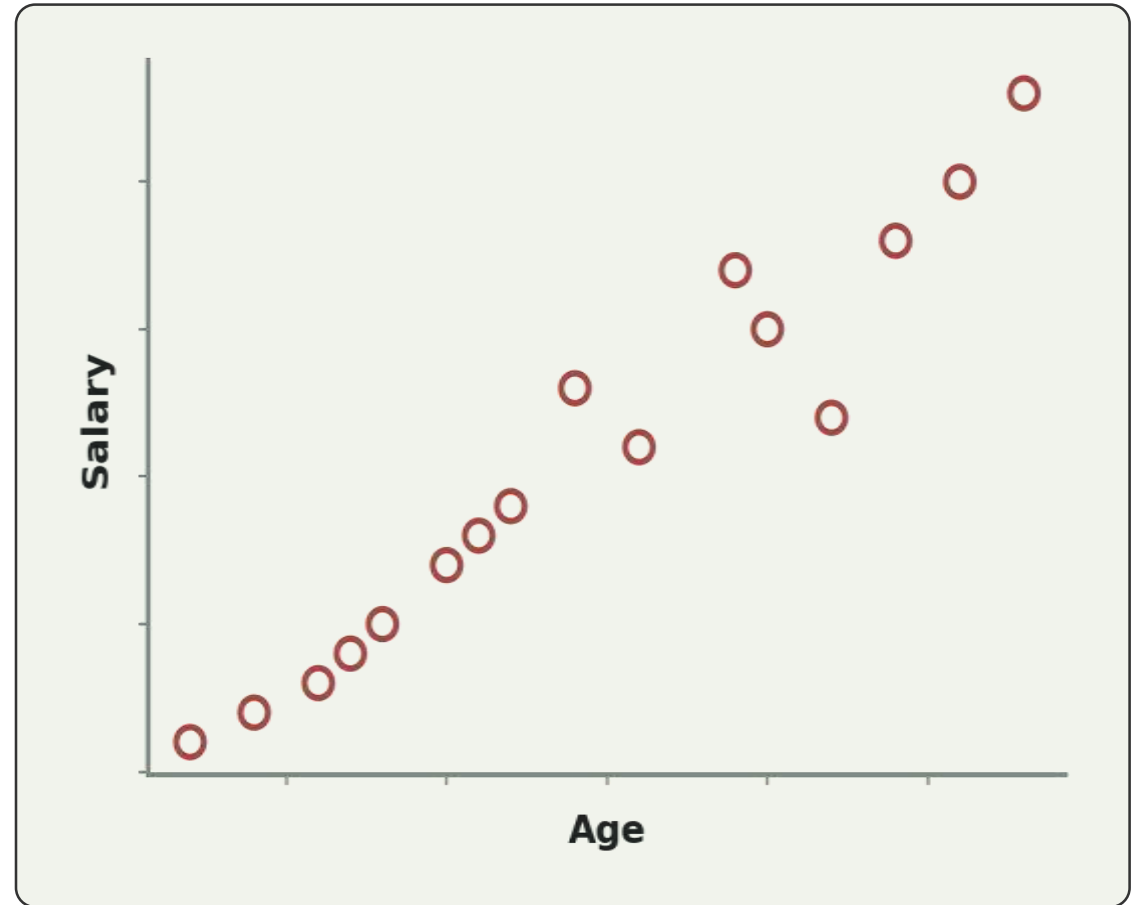
Linear Regression

Fitting the best straight line through your data



The Problem Statement

- Say we have a dataset of Age and Salary for a group of employees.
- Age is the input (feature) — what we already know about a person.
- Salary is the output (target) — what we want to predict for someone new.
- Plotting Age (x-axis) against Salary (y-axis) gives a scatter of points that roughly trends upward.
- The goal: find a line that best represents that trend, so we can predict Salary for any Age.

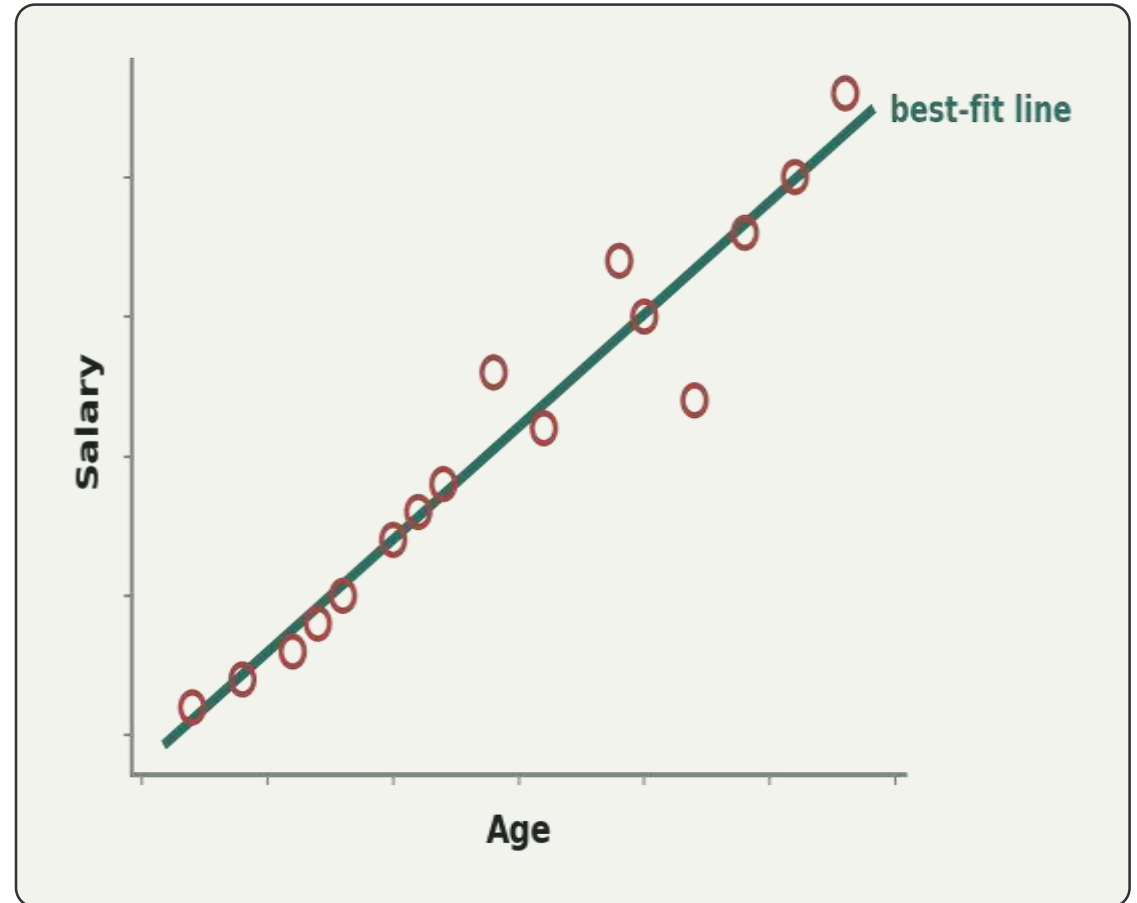


Raw data: each point is one employee's Age and Salary



The Best-Fit Line

- Among every possible line we could draw through the scatter plot, we want the one that lies closest to all the points overall.
- This is called the best-fit line (or regression line).
- It won't pass through every point exactly — real data is noisy — but it captures the overall trend.
- Once we have this line, we can plug in any new Age and read off a predicted Salary.

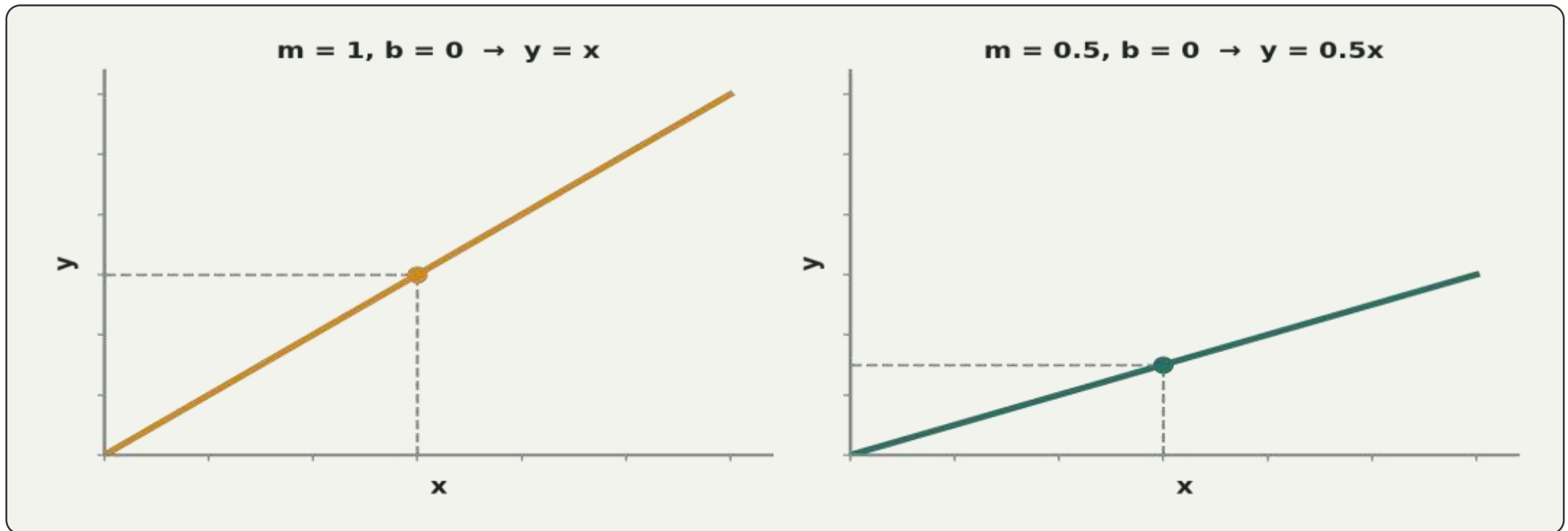


The line that sits closest to every point overall



The Equation of a Line: $y = mx + b$

- Every straight line can be written as $y = mx + b$ — the same equation from school algebra. m is the slope — how steeply the line rises for each step along x . b is the intercept — the value of y when $x = 0$, where the line crosses the y -axis. x is the data point you plug in, and y is the resulting prediction.





Worked Example: Reading the Line

- With slope $m = 1$ and intercept $b = 0$, the line is simply $y = x$.
- With slope $m = 0.5$ and intercept $b = 0$, every prediction is scaled down by half.

```
# y = mx + b
```

```
# Case 1: m = 1, b = 0
```

```
x = 1.5
```

```
y = 1 * x + 0      # y = 1.5
```

```
# Case 2: m = 0.5, b = 0
```

```
x = 1.5
```

```
y = 0.5 * x + 0    # y = 0.75
```

```
# Changing m and b reshapes the line –
```

```
# this is exactly what "training" tunes
```



The ML Notation: $h_{\theta}(x)$

- In ML, we usually write the line as $h_{\theta}(x) = \theta_0 + \theta_1 \cdot x$ instead of $y = mx + b$ — same idea, different symbols.
- θ_0 (theta-zero) is the intercept — same role as b .
- θ_1 (theta-one) is the slope — same role as m .
- h stands for hypothesis — the model's current best guess function.
- Training a Linear Regression model literally means finding the perfect values of θ_0 and θ_1 .



**Goal: find the
best θ_0 and θ_1**



PART 3

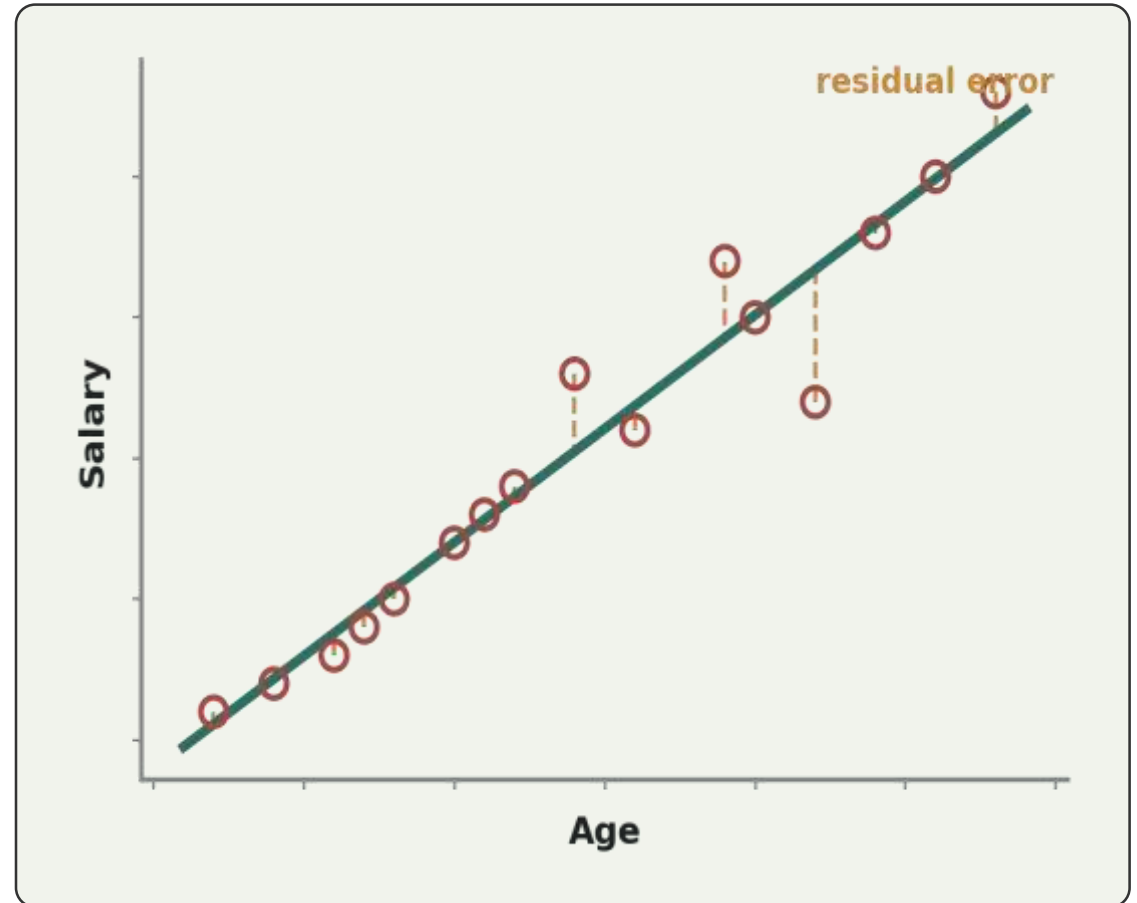
The Cost Function

How do we know if a line is good or bad?



Residual Error

- For every data point, the residual error is the gap between the actual value and what the line predicted.
- Some points sit above the line, some below — the residual can be positive or negative.
- A good line keeps these gaps small overall, across every point in the dataset.
- A bad line has large residuals — its predictions are consistently far from reality.



The dashed segments are residuals — the vertical gap at each point



The Cost Function $J(\theta_0, \theta_1)$ — In Depth

IN PLAIN WORDS: For every point, check how far off the line's guess was, square that miss (so it's always positive and big misses hurt more), add up all those squared misses, then divide by the count so it's a fair average error.

$$J(\theta_0, \theta_1) = \sum_{i=1}^m \frac{1}{2m} \cdot (h(x_i) - y_i)^2$$
$$h(x_i) = \theta_0 + \theta_1 \cdot x_i \quad (\text{the line's guess})$$

WHAT EACH SYMBOL MEANS

m	Total number of training examples (rows) you have.
Σ	"Sum this up" — repeat for every single point, then add the results together.
$h(x_i)$	The line's predicted value for point i — plug x_i into $y = mx + b$.
y_i	The actual, real value for point i from your dataset.
$()^2$	Squaring: makes misses positive, and punishes big misses much harder than small ones.
$1/(2m)$	Averages the total error — the '2' just cancels out neatly later in the calculus.

Smaller J = a better-fitting line. Training the model is just repeatedly adjusting θ_0 and θ_1 to make J as small as possible.



PART 4

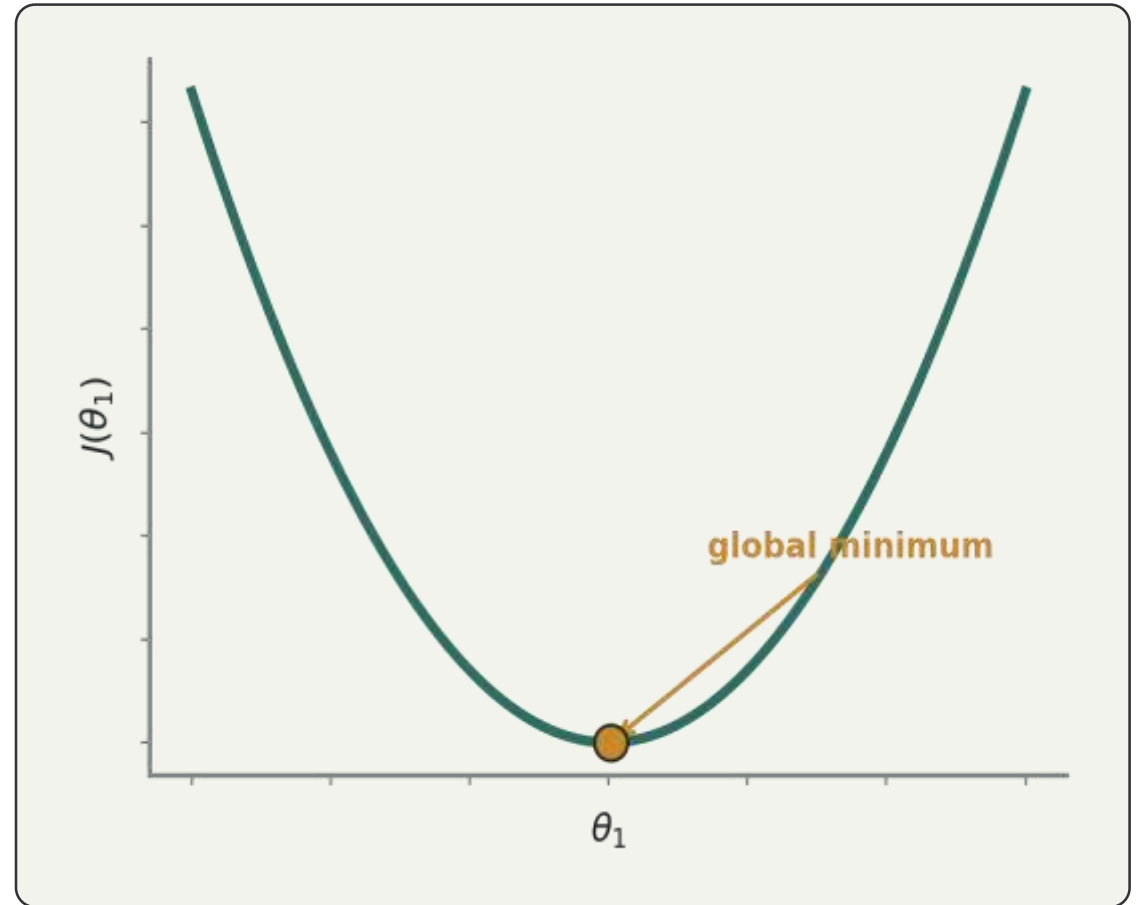
Gradient Descent

The algorithm that finds the minimum-error line



What Is Gradient Descent?

- Gradient Descent is the optimization algorithm that searches for the θ_0 and θ_1 that minimize the cost function J .
- Picture plotting J against θ_1 — it forms a bowl-shaped (convex) curve.
- The lowest point of that bowl is the global minimum — the setting of θ_1 with the smallest possible error.
- Gradient descent starts at some random point on the curve and takes repeated small steps downhill toward that minimum.



$J(\theta_1)$ forms a convex bowl — the bottom is the global minimum



Worked Example: Evaluating a Few θ Values

- Take a tiny dataset: (1,1), (2,2), (3,3) — a perfect line $y = x$.
- Try a few candidate slopes and see how the cost function J reacts.

```
# Data: {(1,1), (2,2), (3,3)},     $\theta_0 = 0$ 

#  $\theta_1 = 1$   -> predictions: 1, 2, 3 (perfect match)
#               $J(\theta_1=1) = 0$                     <- global minimum

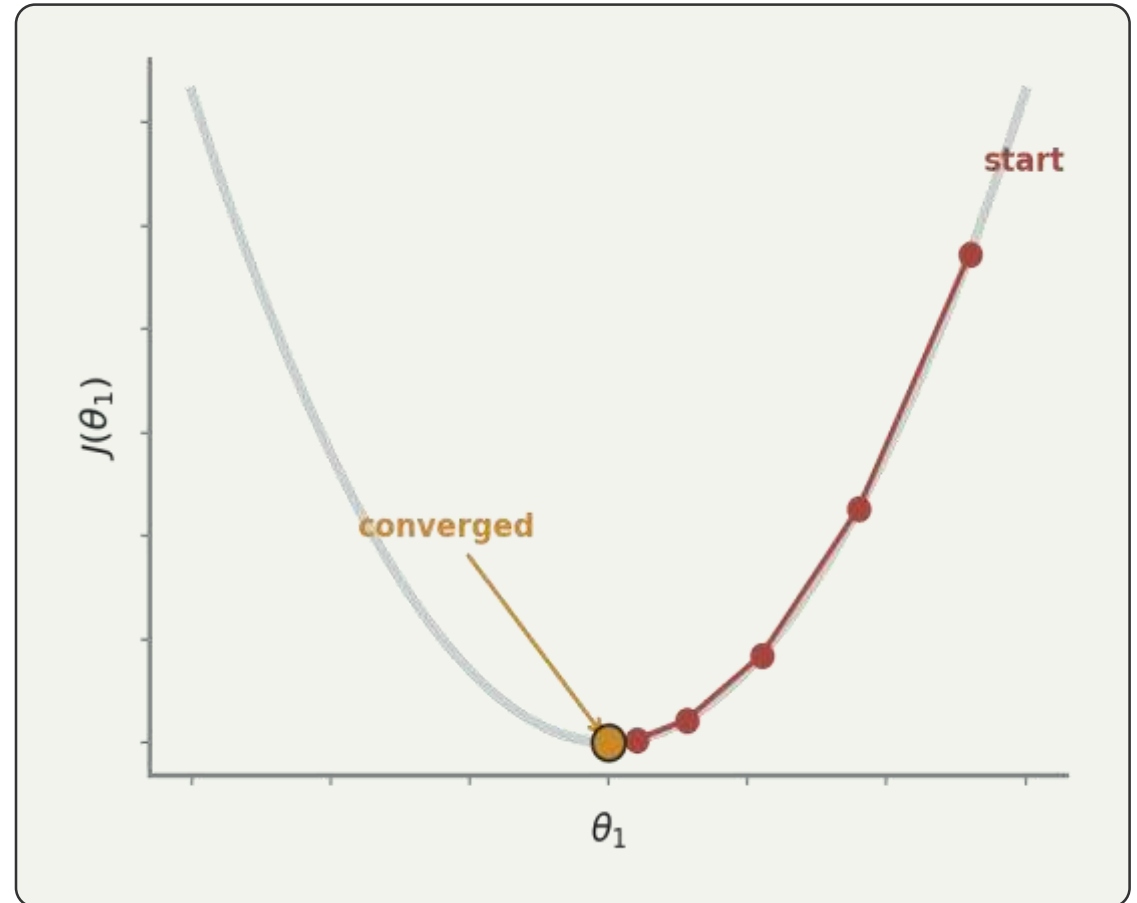
#  $\theta_1 = 0.5$  -> predictions: 0.5, 1, 1.5
#               $J(\theta_1=0.5) \approx 0.58$ 

#  $\theta_1 = 0$    -> predictions: 0, 0, 0
#               $J(\theta_1=0) \approx 2.33$ 
```




Watching Gradient Descent Converge

- Starting from a random θ_1 far from the answer, each step moves closer to the bottom of the bowl.
- Steps are naturally larger when far from the minimum, and shrink automatically as it gets close.
- Once the steps stop making meaningful progress, the algorithm has converged on $\theta_1 = 1$ — the global minimum for this dataset.



Each red dot is one gradient descent iteration, walking downhill



The Gradient Descent Update Rule — In Depth

IN PLAIN WORDS: To take one step downhill, nudge each θ a little in whichever direction makes J smaller — the learning rate α controls how big that nudge is, and the slope (derivative) tells you which direction is 'downhill' from where you're standing.

```
repeat until convergence {  
  
     $\theta_j := \theta_j - \alpha \cdot \partial/\partial\theta_j J(\theta_0, \theta_1)$   
  
} (update  $\theta_0$  and  $\theta_1$  simultaneously)
```

WHAT EACH SYMBOL MEANS

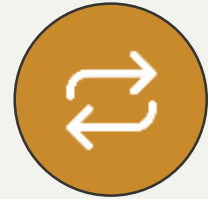
$:=$	"Gets updated to" — replace the old θ_j with this new value.
θ_j	Whichever parameter you're updating — θ_0 or θ_1 .
α	Learning rate — how big a step to take. Small = slow but safe; too big = overshoots.
$\partial/\partial\theta_j J$	The slope of the cost curve at your current θ_j — which way is downhill, and how steep.

Why subtract? If the slope is positive (curve rising to the right), subtracting moves θ_j left — downhill. If the slope is negative, subtracting a negative moves θ_j right — still downhill either way.



Repeat & Convergence Theorem

- At each step, gradient descent nudges θ_0 and θ_1 slightly in the direction that reduces J the fastest.
- It repeats this update over and over — that's the 'Repeat' part of the convergence theorem.
- Convergence means the steps get smaller and smaller as you approach the minimum, until J stops meaningfully decreasing.
- At that point, the algorithm has found (or nearly found) the optimal θ_0 and θ_1 .

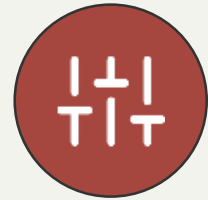


**Repeat until
J stops
decreasing**



The Learning Rate (α)

- The learning rate α controls how big each step downhill is.
- Too small an α — the model learns correctly but painfully slowly, taking many iterations.
- Too large an α — the steps overshoot the minimum and the algorithm can bounce around or even diverge.
- Picking a good α is one of the first 'tuning knobs' you learn to adjust in any ML model.



**α too big →
overshoots
 α too small →
too slow**



PART 5

Multiple Linear Regression

When more than one feature drives the prediction



Multiple Linear Regression — In Depth

IN PLAIN WORDS: Instead of one input driving the prediction, now every feature gets its own weight (θ). The prediction is just the intercept plus each feature multiplied by how much it matters — all added together.

Simple: $h(\theta) = \theta_0 + \theta_1 \cdot x_1$

Multiple: $h(\theta) = \theta_0 + \theta_1 \cdot x_1 + \theta_2 \cdot x_2 + \theta_3 \cdot x_3$
 $+ \dots + \theta_n \cdot x_n$

WHAT EACH SYMBOL MEANS

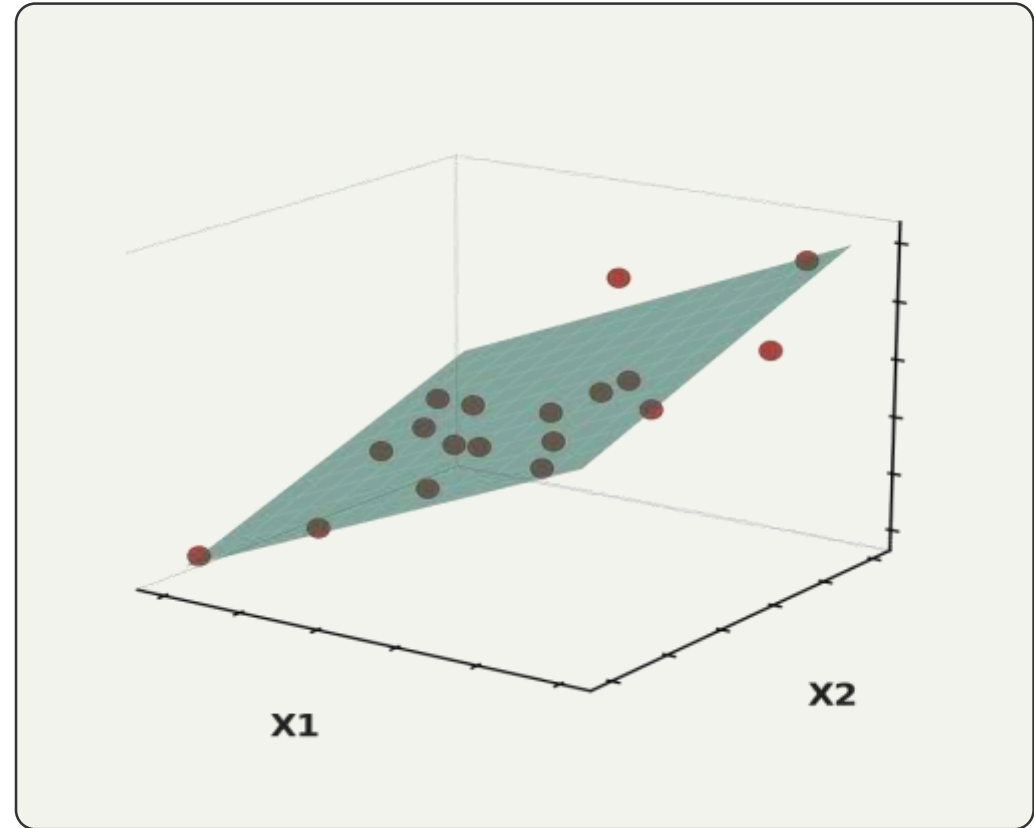
θ_0	The intercept — the baseline prediction when every feature is 0.
$x_1, x_2 \dots x_n$	Each individual feature — e.g. Age, Location, Years of Experience.
$\theta_1, \theta_2 \dots \theta_n$	Each feature's own weight — how much that feature pushes the prediction up or down.
n	The total number of features being used.

Example: predicting Salary from Age, Location, and Experience — each feature gets multiplied by its own θ , then everything is summed into one prediction.



The Hyperplane

- With one feature, the best fit is a line. With two features, it becomes a flat plane cutting through 3D space.
- With three or more features, it's called a hyperplane — the same idea generalized to higher dimensions we can't directly visualize.
- Each axis (x_1 , x_2 , x_3 ...) represents one feature, and the hyperplane still tries to sit as close as possible to every data point.
- The math scales up cleanly — same cost function, same gradient descent, just more θ parameters to solve for.



Two features (x_1 , x_2) predicting Y — a flat plane instead of a line



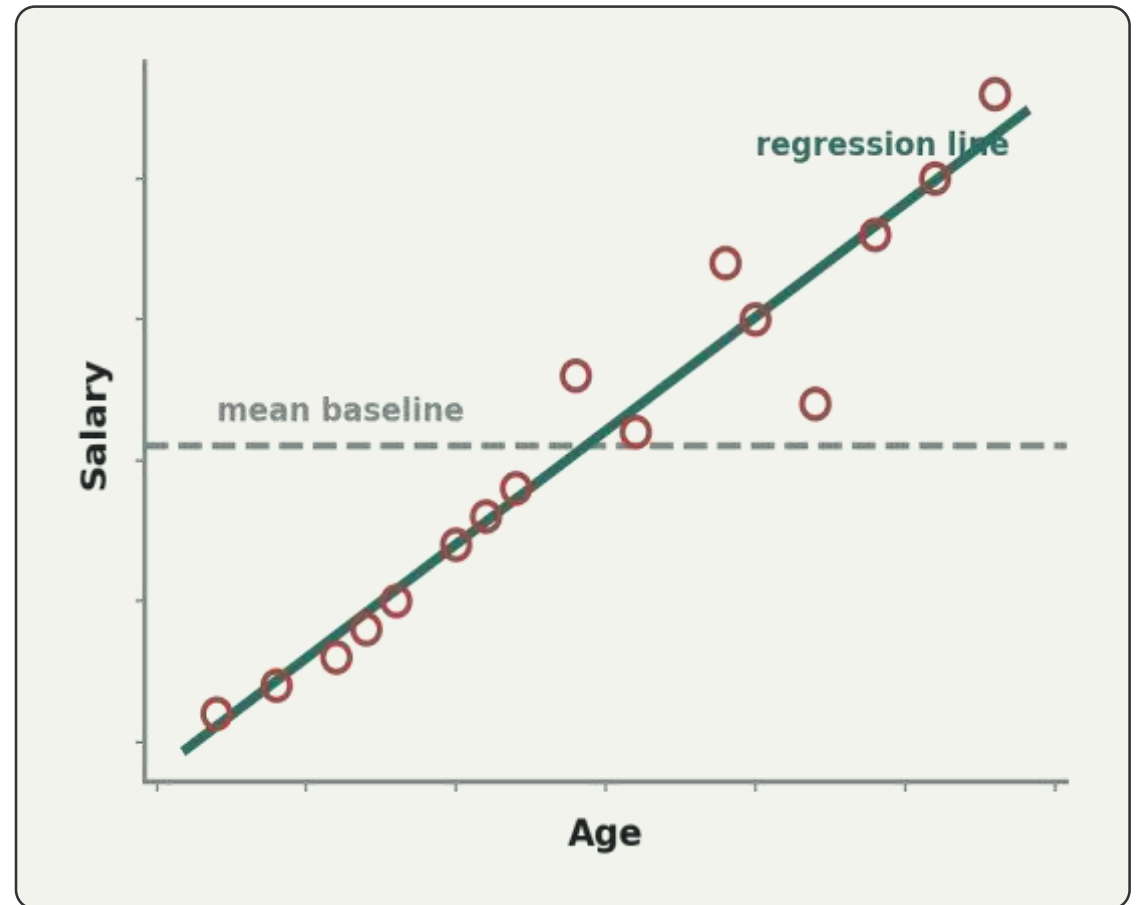
PART 6

Evaluating the Model

Is this line actually any good?

R^2 — The Coefficient of Determination

- R^2 measures how much of the variation in the output is explained by the model, compared to just guessing the average every time.
- The result is always a decimal, usually shown as a percentage — e.g. 0.86 means 86%.
- Higher R^2 generally means a better fit, but it isn't the whole story — see the next slide.



R^2 compares the model's line against just predicting the mean every time



The R² Formula — In Depth

IN PLAIN WORDS: Compare how wrong your model's line is (the residuals) against how wrong you'd be if you just guessed the average every single time. If your model barely beats the average, R² is near 0. If it nails every point, R² is near 1.

$$R^2 = 1 - \frac{\text{Sum of Squared Residuals}}{\text{Sum of Squared Total}}$$
$$= 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

WHAT EACH SYMBOL MEANS

y_i	The actual, real value for point i.
$\hat{y}_i$	("y-hat") — your model's predicted value for point i.
$\bar{y}$	("y-bar") — the plain average of all actual y values, ignoring the model entirely.
Numerator	How wrong your model's predictions are, squared and summed.
Denominator	How wrong just guessing the average would be — the baseline to beat.

R² = 89% means your model explains 89% of the ups and downs in the data — only 11% is left as unexplained error.



The Adjusted R^2 Formula — In Depth

IN PLAIN WORDS: Plain R^2 can be tricked — it goes up (or stays flat) every time you add ANY feature, even a useless one. Adjusted R^2 fixes this by charging a small 'entry fee' per feature, so it only rises if that feature genuinely earns its keep.

$$\text{Adj } R^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

WHAT EACH SYMBOL MEANS

R^2	The plain R^2 score you already calculated for this model.
n	Number of rows (samples/observations) in your dataset.
p	Number of features (predictor columns) the model is using.
$n - p - 1$	Shrinks as you add features — this is literally the 'fee' for each new column.

Example: adding a weak 'sleep hours' feature — R^2 rises 81%→88%→89%, but Adjusted R^2 goes 79%→89%→86%. That drop is the tell that the last feature wasn't worth adding.



PART 7

Overfitting vs. Underfitting

The two ways a model can go wrong



Overfitting vs. Underfitting

Overfitting

- Model memorizes training data too closely
- Training performance: excellent
- Testing performance: poor
- Low bias, high variance
- Fix: simplify the model, get more data, regularize

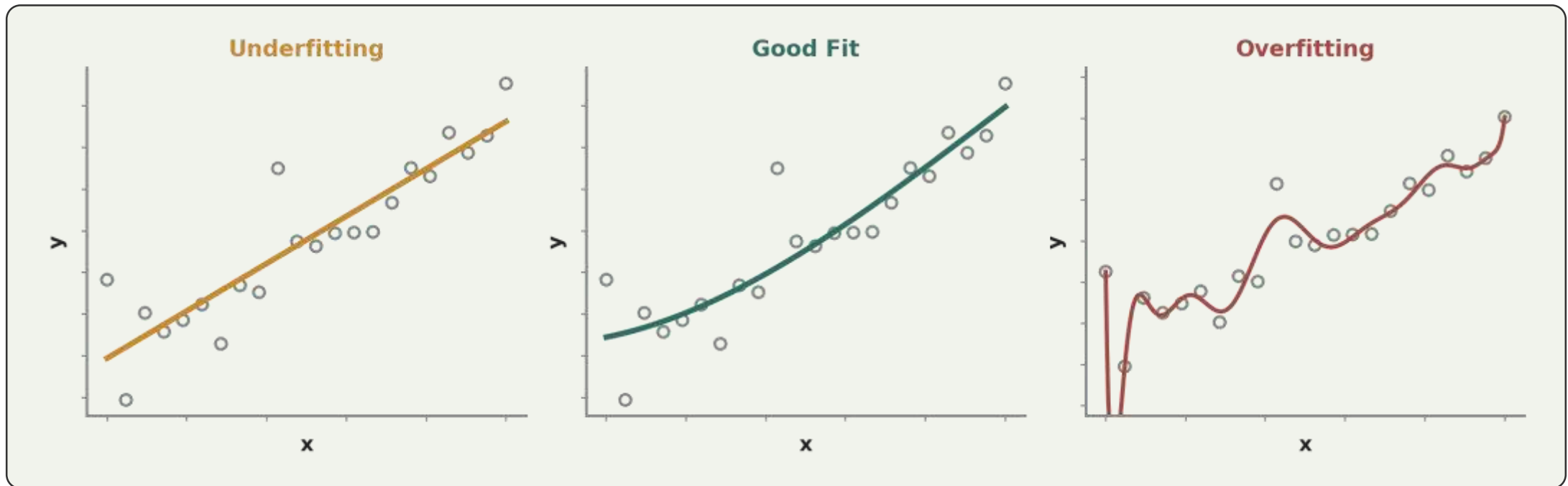
Underfitting

- Model is too simple to capture the pattern
- Training performance: poor
- Testing performance: also poor
- High bias, high variance
- Fix: add features, use a more complex model



What Overfitting & Underfitting Actually Look Like

- Underfitting — a straight line forced onto curved data; it misses the pattern in both training and test data alike.
- Good Fit — a smooth curve that follows the true trend without chasing every bit of noise.
- Overfitting — a wiggly curve that hits nearly every training point exactly, but that wiggle won't generalize to new data.



Same data, three different model complexities



Reading Train vs. Test Accuracy



Model M1

Train: 92%
Test: 64%
→ Overfitting



Model M2

Train: 92%
Test: 90%
→ Perfect fit
(low bias, low variance)



Model M3

Train: 68%
Test: 64%
→ Underfitting



PART 8

Ridge & Lasso Regression

Taming overfitting by penalizing complexity



The Ridge Regression Formula — In Depth

IN PLAIN WORDS: Take the normal cost function (average squared error) and tack on an extra penalty that grows if any slope (θ) gets too large. To keep the total cost low, the model is forced to keep its θ values small and reasonable — instead of stretching to fit every last wiggle in the training data.

$$J(\theta) = \underbrace{\sum_{i=1}^m \frac{1}{2m} \cdot (h(x_i) - y_i)^2}_{\text{usual prediction error}} + \underbrace{\lambda \cdot (\theta_1)^2}_{\text{the new penalty}}$$

WHAT EACH SYMBOL MEANS

λ	("lambda") — how strong the penalty is. $\lambda = 0 \rightarrow$ plain Linear Regression. Bigger $\lambda \rightarrow$ smaller, more conservative θ .
$(\theta_1)^2$	Squaring the slope — same trick as before, always positive, and big slopes get punished hard.
Left term	The same MSE cost function you already know — how wrong the predictions are.
Right term	The 'complexity tax' — added purely to discourage relying too heavily on any one feature.

Lasso uses the exact same idea, but penalizes $|\theta_1|$ (absolute value) instead of $(\theta_1)^2$ — that small change is what lets Lasso shrink some coefficients all the way to zero.



Ridge vs. Lasso — What's the Difference?

Ridge (L2)

- Cost = $\text{MSE} + \lambda \cdot (\text{slope})^2$
- Shrinks all coefficients toward zero
- Rarely sets any coefficient to exactly zero
- Good when most features matter a little

Lasso (L1)

- Cost = $\text{MSE} + \lambda \cdot |\text{slope}|$
- Can shrink weak coefficients to exactly zero
- Effectively performs feature selection
- Good when only a few features truly matter



Try It Yourself — Practice Tasks

Use any small numeric dataset (e.g. house prices, salary data) and work through these with scikit-learn

1

Load the dataset and split it: `X_train`, `X_test`, `y_train`, `y_test`

2

Fit a `LinearRegression()` model on the training data

3

Print the learned coefficients (θ) and intercept

4

Predict on `X_test` and compare against `y_test`

5

Calculate R^2 using `model.score()` or `r2_score()`

6

Add a second feature and watch R^2 vs Adjusted R^2 diverge

7

Compare train vs test accuracy to check for over/underfitting

8

Try `Ridge()` and `Lasso()` and compare their coefficients



RECAP

Linear Regression is a straight line, fit by shrinking a cost function with gradient descent — everything else refines that idea.

Next session: Logistic Regression — predicting categories instead of numbers